Rei Thao

(530)329-3050 ◊ rei.t.thao@gmail.com ◊ www.linkedin.com/in/rei-t-thao ◊ github.com/rthao1

Education

University of the Pacific - Stockton *Stockton, CA*

Bachelor of Science, Computer Science *Expected Graduation: May 2026*

◊ Cumulative GPA: 3.60 / Computer Science GPA: 3.76

Related Coursework

◊ Software Engineering
◊ Design/Analysis of Algorithms
◊ System Admin and Security
◊ Computer Network Security
◊ Cybersecurity Threat Hunting
◊ Computer Networking
◊ Database Management Systems (IP)
◊ Programming Languages
◊ Computer Systems and Networks
◊ Operating Systems
◊ Artificial Intelligence
◊ Data Structures
◊ Data Analytics Programming (IP)

Projects

Threat Hunt Simulation *Fall 2025*

› *Windows Server vulnerability assessment in team using Wazuh SIEM and Sysmon to hunt for suspicious persistence and aligned activity to MITRE ATT&CK Framework*
› *Delivered a security report with IOC findings, threat detection, log analysis, detection logic, and remediation recommendations*

Penetration Test Simulation *Spring 2025*

› *Conducted penetration test of fictitious organization Tiger Enterprises' VM via Nmap*
› *Exploited system and exfiltrated user log in data using Kali Linux and Metasploit*

Single Cycle Processor Simulator *Fall 2024*

› Developed Python-based MIPS simulator that executes a subset of MIPS instructions

Machine Language Converter *Fall 2024*

› Created Python program that translates MIPS assembly code to MIPS machine code

Relaxation Hub Application *Spring 2024*

› Collaborated in a group of four students to develop a Java-based video game

Syntactic Analysis Program *Fall 2023*

› Coded scanner, parser, and evaluator functions using AST trees and dictionaries

Skills

◊ Linux OS
◊ C/C++
◊ Metasploit
◊ Nessus
◊ ManageEngine
◊ T-Pot
◊ Okta
◊ Windows OS
◊ Python
◊ Ubuntu
◊ Penetration Testing
◊ Microsoft Office 365
◊ MITRE ATT&CK
◊ Active Directory (AD)
◊ Eclipse
◊ Java
◊ Kali
◊ Threat Hunting
◊ Scrum
◊ Sysmon
◊ MacOS
◊ GitHub
◊ Haskell
◊ ServiceNow
◊ Wireshark
◊ Wazuh SIEM
◊ NIST 800-171

Experience

University of the Pacific, Pacific Technology *Stockton, CA*

Help Desk, Student Technician I *February 2024 – Present*

› Provide tier I/II Windows & macOS support to 15,000+ users in enterprise environment
› Manage and document 20+ tickets weekly using ServiceNow and ManageEngine, ensuring accurate and timely resolution
› Image, surplus, and maintain devices while maintaining compliance with IT policies

Theta Tau Professional Engineering Fraternity *Stockton, CA*
Vice President *December 2025 – Present*

› Managed bylaw governance and policy updates, ensuring compliance and identifying areas for procedural improvement
› Coordinated committees and delegated tasks to maintain accountability and alignment with organizational policies

Theta Tau Professional Engineering Fraternity *Stockton, CA*
Recruitment Lead *May 2024 – December 2024*

› Lead group of thirty individuals to host community service and professional events
› Communicate and interact with sixty individuals

Cinemark *Yuba City, CA*
Theatre Team Member *October 2021 – August 2022*

› Fulfill customer support needs in fast-paced environment
› Handle customer transactions

Other Affiliations

◊ Theta Tau Professional Engineering Fraternity
◊ MESA – Math, Engineering, Science, Achievement
◊ Dean's Honor Roll – Fall 2022, Fall 2023, Fall 2024

Rei Thao
(530)329-3050 ◊ rei.t.thao@gmail.com ◊ www.linkedin.com/in/rei-thao/

Professional References

Rebeca Noble Direct Supervisor at University of the Pacific
Assistant Director of Customer Service Delivery
Pacific Technology
UNIVERSITY OF THE PACIFIC, 3601 Pacific Avenue
Office: 209.946.7664
Mobile: 209.662.4729
rnoble@pacific.edu

Cybersecurity Threat Hunting Technical Paper *Fall 2025*

1. **Overview**

   From August to December 2025, I conducted a full-scope threat hunting engagement for a fictious Ace Tomato Company, analyzing six critical assets including two domain controllers, two Windows endpoints, a LAMP web server, and an unknown Kali Linux host. Using Wazuh SIEM, Sysmon, and the MITRE ATT&CK framework, I identified multiple active compromises, privilege escalation attempts, and persistence mechanisms across the environment.

   This project demonstrates my ability to perform end-to-end threat hunting, correlate multi-source telemetry, identify attacker behavior, and produce actionable remediation guidance.

Key Responsibilities

› Performed log analysis and threat hunting across Windows and Linux systems
› Investigated suspicious authentication, group policy modifications, and service creation
› Identified indicators of compromise (IoCs) and mapped activity to MITRE ATT&CK
› Assessed risk severity and potential business impact
› Delivered a full remediation plan including containment, credential resets, and hardening
› Authored a comprehensive 10-page threat hunting report

Major Findings

1. **Privileged Account Manipulation (Critical)**
   › Unauthorized modification of **Enterprise Admins** and **Schema Admins** groups
   › "ANONYMOUS LOGON" added to high-privilege groups
   › Administrator account removed from critical groups
   › Clear evidence of **persistence**, **defense evasion**, and **domain compromise**
2. **Malicious Windows Service Creation (Critical)**
   › 14 unauthorized services created across both domain controllers
   › Randomized service names and binaries placed in %systemroot%
   › All services executed under **LocalSystem**, enabling full system control
   › Repeated crashes of win32calc.exe suggesting **process injection** or malicious persistence
3. **Shellshock Exploitation Attempts on LAMP Server (High)**
   › Repeated GET requests for sensitive files (e.g., CustomerEmail.txt)
   › All requests originated from unknown host **10.0.0.200**
   › Wazuh flagged multiple **Shellshock exploitation attempts**
   › Potential for **remote code execution** and **lateral movement**

Tools & Frameworks Used

› Wazuh SIEM — log aggregation, correlation, and alerting
› Sysmon (EDR) — process creation, service execution, and network telemetry
› MITRE ATT&CK — behavioral mapping and classification
› NIST CSF — structured approach to identify, detect, respond, and recover

Recommendations

1. **Immediate**

› Isolate DC1 and DC2
› Block traffic from 10.0.0.200
› Remove unauthorized group memberships
› Reset all privileged credentials
› Remove malicious services and validate registry integrity
› Patch Bash to mitigate Shellshock

2. **Long-Term**
   › Enforce MFA for all admin accounts
   › Strengthen password and lockout policies
   › Increase monitoring of startup folders, Run keys, and scheduled tasks
   › Improve incident response documentation and staff training

Computer Network Security Penetration Test Report ***Spring 2025***

1. **Overview**
   Tiger Enterprises, Inc., a fictitious consulting company, requested a full penetration test of their "U" server after operating without a system administrator for three years. I conducted a full assessment including reconnaissance, vulnerability scanning, exploitation, privilege escalation, and credential harvesting. The engagement successfully identified multiple critical vulnerabilities and resulted in full system compromise.

   This project demonstrates my ability to perform structured penetration testing, identify real-world attack paths, and communicate findings clearly to both technical and non-technical audiences.

Key Responsibilities

› Performed reconnaissance and service enumeration using Nmap
› Identified vulnerable services and mapped them to known exploits
› Executed remote code execution and privilege escalation attacks
› Conducted post-exploitation activities including credential harvesting and database access
› Validated user credentials and confirmed system compromise
› Delivered remediation recommendations and security hardening guidance

Major Findings

1. **Remote Code Execution via ProFTPD mod_copy (Critical)**
   › **Vulnerable Service:** ProFTPD 1.3.5 with mod_copy enabled
   › **Exploit Used:** exploit/unix/ftp/proftpd_modcopy_exec (Metasploit)
   › **Impact:** Remote shell access as www-data
   › **Root Cause:** Outdated FTP service with known RCE vulnerability
   › This exploit provided the initial foothold into the system.
2. **Privilege Escalation via PwnKit (CVE-2021-4034) (Critical)**
   › **Vulnerable Component:** pkexec (polkit)
   › **Exploit Used:** PwnKit local privilege escalation
   › **Impact:** Full root access from a low-privilege shell
   › **Discovery Method:** LinPEAS enumeration
   › This allowed complete control of the host and access to all system resources.
3. **Credential Discovery & Sensitive Data Exposure (High)**
   Using **LinPEAS**, I discovered:
   › MySQL root credentials in /etc/mysql/debian.cnf
   › Plaintext database credentials in multiple PHP configuration files
   › Full user list and passwords stored in the payroll.users table
   › Valid login credentials for user **kylo_ren** (Daddy_Issues2)
4. **Impact:**
   › Unauthorized database access
   › Exposure of sensitive payroll information
   › Ability to authenticate as real system users

Post-Exploitation Achievements

› Gained root shell
› Extracted MySQL database contents

› Verified plaintext user passwords
› Logged in as real system user
› Identified dozens of vulnerabilities via Nessus (72 total)
› This demonstrated full compromise of confidentiality, integrity, and availability.

Recommendations

1. **Immediate**
   › Upgrade ProFTPD to a secure version
   › Patch PwnKit vulnerability across all Linux hosts
   › Reset all user and database passwords
   › Remove plaintext credentials from configuration files
   › Restrict MySQL access and enforce least privilege
2. **Long-Term**
   › Implement centralized system administration
   › Enforce password policies and MFA
   › Deploy regular vulnerability scanning
   › Harden web server and database configurations
   › Improve credential storage practices

Single Cycle Processor Simulator *Fall 2024*

› This project is a Python-based MIPS simulator that is able to execute a subset of MIPS machine instructions.

Project Repository

› https://github.com/rthao1/SingleCycleProcessorSimulator

Key Functions and Code

```
def convert_Control(line): #assigns control
    combined_control = ''
    register = ''
    opcode = line[:6]
    rs = line[6:11]
    rt = line[11:16]
    rd = line[16:21]
    shamt = line[21:26]
    funct = line[26:]
    immediate = line[16:]

    if opcode == "000000":  # R-type (add, sub)
        Control_Signals["RegDst"] = 1
        Control_Signals["ALUSrc"] = 0
        Control_Signals["RegWrite"] = 1

        if funct == "100000":  # 'add' funct code
            Control_Signals["ALUOp1"] = 1
            Control_Signals["ALUOp2"] = 0
            Registers[int(rd,2)] = Registers[int(rs,2)] + Registers[int(rt,2)] #stores add result in register

        elif funct == "100010":  # 'sub' funct code

            Control_Signals["ALUOp1"] = 1
            Control_Signals["ALUOp2"] = 0
            Registers[int(rd,2)] = Registers[int(rs,2)] - Registers[int(rt,2)] #stores sub result in register

    elif opcode == "001000":  # I-type (addi)
        Control_Signals["RegDst"] = 0
        Control_Signals["ALUSrc"] = 1
        Control_Signals["RegWrite"] = 1
        Control_Signals["ALUOp1"] = 0
        Control_Signals["ALUOp2"] = 0
        Registers[int(rt,2)] = Registers[int(rs,2)] + BitArray(bin=immediate).int #stores addi result in register

    for value in Control_Signals.values():
        combined_control += str(value)
    for value in Registers.values():
        register += str(value) + '|'
    return combined_control, register.strip('|') #returns register and control text lines
```

Machine Language Converter *Fall 2024*

› This Python program, written in VS Code, converts MIPS assembly into machine language. It receives a set of instructions, in MIPS assembly, and checks dictionaries to see if each instruction is an I or R type encoding problem. It then checks what type of register is being called in the statement and makes the correct conversion based on the result. Lastly, it checks what is being called again to correctly place the conversions.

Project Repository

› https://github.com/rthao1/MachineLanguageConverter

Key Functions and Code

```
def convertRegister(register):
    if register[0] == "$":
        register = register[1:]
    if register == '0': return "00000"
    if register == '1': return "00001"
    return f"{int(register):05b}"

def convertOffset(offset):
    offset = int(offset)
    if offset == 0: return "0000000000000000"
    if offset < 0: return f"{(65536 + offset) & 0xFFFF:016b}"
    return f"{offset:016b}"
```

◊ **convertRegister**

› Reads MIPS assembly input and converts instruction to machine language to appropriate register

◊ **convertOffset**

› Reads offset provide in MIPS assembly and converts the offset to correct 16-bit number.

```
if opcode in R_Code:
    function = R_Code[opcode]
    if opcode == 'srl' or opcode == 'sll':
        rd = convertRegister(rd)
        shamt = f"{int(rt):05b}"
        rt = convertRegister(rs)
        rs = "00000"
        return f"000000{rs}{rt}{rd}{shamt}{function}"
    else:
        opcode = "000000"
        shamt = "00000"
        if rs[:3] in Registers: rs = convertIRegister(rs)
        else: rs = convertRegister(rs)

        if rd[:3] in Registers: rd = convertIRegister(rd)
        else: rd = convertRegister(rd)

        if rt[:3] in Registers: rt = convertIRegister(rt)
        else: rt = convertRegister(rt)

        return f"{opcode}{rs}{rt}{rd}{shamt}{function}"
```

◊ **convertInstruction**

› Reads the entire MIPS assembly instruction and assigns registers correct values according to the function called using arrays and previous functions.

Relaxation Hub Application *Spring 2023*

› The objective of this project, written in Eclipse and GitHub, was to create a game that allows users to relax through a variety of mini games. Our goals were to create a game that was intuitive, easy to use, engaging, and helpful to alleviate stress and anxiety.

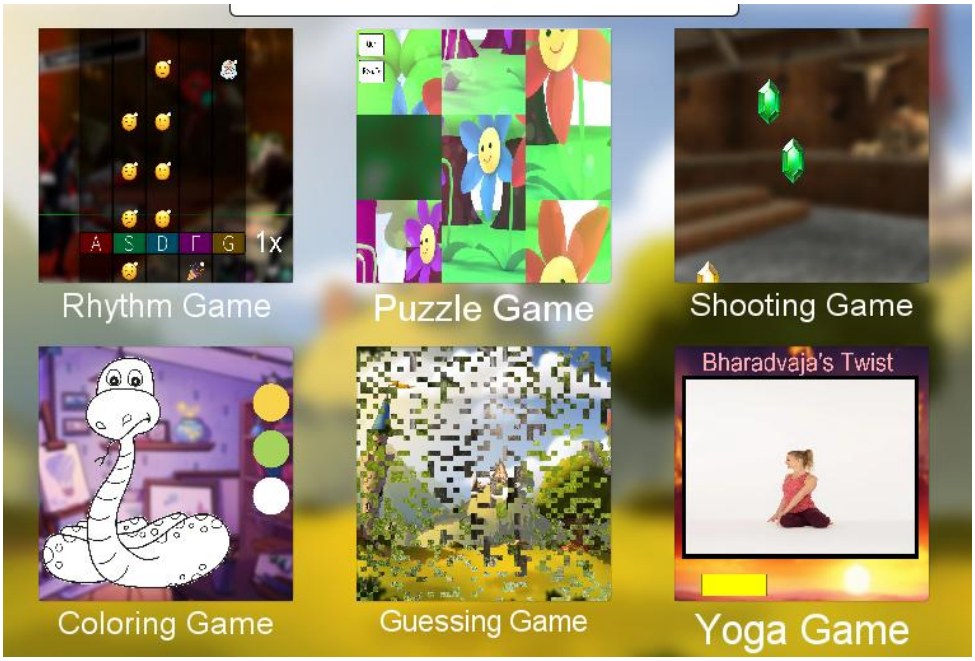
Project Repository

› https://github.com/comp129/customer-2024-entei-2024

Key Functions and Code

```
public Game(String name, Float depressed, Float anxious, Float angry, MainApplication app) {
    moods = new HashMap<>();
    this.preview = NewGImage.create("pics/gamepreview/" + name.replaceAll("\\s+", "") + ".png");
    this.name = name;
    this.app = app;
    this.moods.put("Anxious", anxious);
    this.moods.put("Angry", angry);
    this.moods.put("Depressed", depressed);
    button = new GButton(name, 0, 0, 200, 200);
    button.setImage(preview);
    button.setColor(Color.WHITE);
    button.addButtonListener((button, e) -> app.switchToMinigame(name));
    add(button);
}
```

› Using data structures, such as hashmaps, and specific questions with mood weights, the program is able to calculate the user's mood and recommend the appropriate minigame.



Syntactic Analysis Program *Fall 2023*

› This project, written in Replit, uses three classes, "Scanner", "Parser", and "Evaluator" to read a text file of a set of code instructions to do as the class names suggest, scan, parse, and evaluate them.

Project Repository

› https://github.com/rthao1/SyntacticAnalysisProgram

Key Functions and Code

```python
def print_ast(node, indent='', output_file=None):
  if output_file is None:
    output_file = sys.stdout
  if isinstance(node, dict):
    for key, value in node.items():
        if isinstance(value, dict):
            print_ast(value, indent + '    ', output_file)
        else:
            print(f"{indent}{key} {value}", file=output_file)
  elif isinstance(node, list):
    for item in node:
        print_ast(item, indent, output_file)
  else:
        print(f"{indent}{node}", file=output_file)
```

◊ **print_ast**

› Recursive function created to print an abstract syntax tree (AST) that consists of dictionaries and lists

```python
def tokenize(input_line):
  tokens = []
  while input_line:
    # Check for whitespace at the beginning of the line and remove it
    match = re.match(r'\s+', input_line)
    if match:
        input_line = input_line[len(match.group(0)):]

    # Check for NUMBER
    match = re.match(r'\d+', input_line)
    if match:
        tokens.append(Token('NUMBER', match.group(0)))
        input_line = input_line[len(match.group(0)):]
        continue

    # Check for KEYWORD
    match = re.match(r'if|then|else|endif|while|do|endwhile|skip', input_line)
    if match:
        tokens.append(Token('KEYWORD', match.group(0)))
        input_line = input_line[len(match.group(0)):]
        continue

    # Check for IDENTIFIER
    match = re.match(r'[a-zA-Z][a-zA-Z0-9]*', input_line)
    if match:
        tokens.append(Token('IDENTIFIER', match.group(0)))
        input_line = input_line[len(match.group(0)):]
        continue

    # Check for SYMBOL
    match = re.match(r'\+|-|\*|/|\(|\)|:=|;', input_line)
    if match:
        tokens.append(Token('SYMBOL', match.group(0)))
        input_line = input_line[len(match.group(0)):]
        continue
    # If no valid token is found, raise an error
    raise SyntaxError("Invalid token")

  return tokens
```

◊ **tokenize**

› Converts a source code string into tokens for evaluation in a compiler

› Identifies invalid tokens preventing incorrect parsing

› Differentiates between numbers, keywords, identifiers, and symbols

› Topic: Quantum Programming and Game Development Languages: A Comparative Analysis of Python and C++

Key Topics and Methods

› Use of Python in Quantum Programming
› Use of C++ in Game Development
› Comparison of C++ and Python in Quantum Programming
› Comparison of C++ and Python in Game Development
› Analyzation of Python and C++ on both high and low levels

Paper Preview

4. COMPARISON OF C++ AND PYTHON IN QUANTUM PROGRAMMING

This section reports on the differences between Python and C++ in quantum programming. An analysis of the functionality of C++ in quantum programming will also be made.

4.2. EXAMINATION OF MAJOR KEY ADVANTAGES HELD BY PYTHON IN QUANTUM COMPUTING

As previously mentioned in section two, Python is heavily favored by many quantum programmers for many good reasons. This includes its user-friendliness, time efficiency, and extensive libraries. In the realm of quantum programming, these factors are some of, if not, the most important aspects to judge when choosing a language to program in. Having to deal with the abundance of abstract and intricate mechanisms that this field has to offer makes it important to the programmer that they need to be able to deeply understand the code and programs and produce them quickly. Thus, Python is the perfect choice to fill this criterion.

4.2. COMPARISON OF C++ FUNCTION QUANTUM PROGRAMMING

In comparison, C++ does not offer the libraries that Python does, such as Django NumPy, Pandas, TensorFlow, and PyTorch. This makes it harder and more time-consuming to do the same tasks that would otherwise be faster to complete in Python. However, C++ makes up for this in performance, control, and memory management, all of which help it make for a good alternative language to pick. Ultimately, Python takes the edge, which may partly be to pre-existing quantum programs and functions improving user experience. In other words, while C++ offers good performance, Python will help the programmer finish the task at hand faster. Today, developer time is valued much higher than machine run-times.

January 8, 2025

Rei Thao
2322 Oregon Way
Yuba City, CA 95991-8475

Dear Rei,

I am very pleased to inform you that you been named to the Dean's Honor Roll for the Fall 2024 Semester. To be eligible for Dean Honor Roll, you must be in good academic standing and have achieved a GPA of 3.5 or higher.

You should be proud of your achievements. Your outstanding scholarship contributes to the high reputation for academic excellence enjoyed by your program, the School of Engineering and Computer Science and the University. Congratulations on this achievement and best wishes for continued success in your academic career.

Sincerely,

Shelly Gulati
Assistant Dean
Book Meeting with Dr. Gulati

Academic Transcript *Fall 202*

Display Transcript

989436309 Rei T. Thao
Feb 16, 2025 12:01 am

This is **not an official** transcript.

Courses which are in progress may also be included on this transcript.

CODE LEGEND
TR = Accepted Transfer Course with C or higher.
TN = Not Accepted Transfer Course.
I = Included units in Earned Hours, in GPA-Hours (letter-graded units) and GPA Points.
E = Excluded units from Earned Hours, GPA Hours and GPA Points.
A = Excluded units from Earned Hours, included units in GPA Hours and GPA Points.

Printing Tip (Printing will only work in Internet Explorer)
To condense transcripts to 2 pages, select Print and click on Properties/Preferences, then select the Finishing Tab and choose 2 pages per sheet.

Transfer Credit Institution Credit Transcript Totals Courses in Progress

Transcript Data

STUDENT INFORMATION

Curriculum Information

Current Program
College: Sch of Engineering & Comp Sci
Major and Department: Computer Science, Computer Science

***Transcript type:ADVS is NOT Official ***

DEGREES AWARDED

Sought: B.S. in Computer Science **Degree Date:**

Curriculum Information

Primary Degree
Major: Computer Science

	Attempt Hours	Passed Hours	Earned Hours	GPA Hours	Quality Points	GPA
Institution:	77.000	77.000	77.000	77.000	282.50	3.66

TRANSFER CREDIT ACCEPTED BY INSTITUTION -Top-

AP Exam: Advanced Placement Test

Subject	Course	Title	Grade	Credit Hours	Quality Points	R
MATH	051	AP Calculus I (4)	TR	4.000		0.00
TRAN	XXX	Non Transferable (AP-1)	TN	0.000		0.00
TRAN	XXX	Non Transferable (AP-2)	TN	0.000		0.00
TRAN	XXX	Non Transferable (AP-3)	TN	0.000		0.00
TRAN	XXX	Non Transferable(AP-1)	TN	0.000		0.00
TRAN	XXX	Non Transferable(AP-1)	TN	4.000		0.00

	Attempt Hours	Passed Hours	Earned Hours	GPA Hours	Quality Points	GPA
Current Term:	4.000	4.000	4.000	0.000	0.00	0.00

Unofficial Transcript

INSTITUTION CREDIT -Top-

Term: Fall 2022
College: Sch of Engineering & Comp Sci
Major: Computer Science
Academic Standing: Good Standing
Additional Standing: Dean's Honor Roll

Subject	Course	Level	Title	Grade	Credit Hours	Quality Points	R
COMP	051	UG	Intro. to Computer Science	A	4.000	16.00	
CORE	001	UG	Problem Solving & Oral Comm	A-	3.000	11.10	
ENGR	010	UG	Dean's Seminar	A	1.000	4.00	
MATH	037	UG	Intro Stats & Prob, plus lab	B+	4.000	13.20	

Term Totals (Undergraduate)

	Attempt Hours	Passed Hours	Earned Hours	GPA Hours	Quality Points	GPA
Current Term:	12.000	12.000	12.000	12.000	44.30	3.69
Cumulative:	12.000	12.000	12.000	12.000	44.30	3.69

Unofficial Transcript

Term: Spring 2023
College: Sch of Engineering & Comp Sci
Major: Computer Science
Academic Standing: Good Standing
Additional Standing: Dean's Honor Roll

Subject	Course	Level	Title	Grade	Credit Hours	Quality Points	R
COMP	047	UG	Discrete Math for Comp Science	A	4.000	16.00	
COMP	053	UG	Data Structures	A	4.000	16.00	
CORE	002	UG	Writing and Critical Thinking	A	4.000	16.00	
ECON	053	UG	Introductory Microeconomics	A	4.000	16.00	

Term Totals (Undergraduate)

	Attempt Hours	Passed Hours	Earned Hours	GPA Hours	Quality Points	GPA
Current Term:	16.000	16.000	16.000	16.000	64.00	4.00
Cumulative:	28.000	28.000	28.000	28.000	108.30	3.86

Unofficial Transcript

Term: Fall 2023
College: Sch of Engineering & Comp Sci
Major: Computer Science
Academic Standing: Good Standing
Additional Standing: Dean's Honor Roll

Subject	Course	Level	Title	Grade	Credit Hours	Quality Points	R
COMP	055	UG	Application Development	A	4.000	16.00	
COMP	141	UG	Programming Languages	B+	4.000	13.20	
GERM	011A	UG	First Year German, 1st Sem.	A	4.000	16.00	
PHYS	017	UG	Concepts of Physics	A	4.000	16.00	

Term Totals (Undergraduate)

	Attempt Hours	Passed Hours	Earned Hours	GPA Hours	Quality Points	GPA
Current Term:	16.000	16.000	16.000	16.000	61.20	3.82

Cumulative:	44.000	44.000	44.000	44.000	169.50	3.85

Unofficial Transcript

Term: Spring 2024

College: Sch of Engineering & Comp Sci
Major: Computer Science
Academic Standing: Good Standing

Subject	Course	Level	Title	Grade	Credit Hours	Quality Points	R
CHEM	023	UG	Elements of Chemistry	C+	4.000	9.20	
COMP	129	UG	Software Engineering	A	4.000	16.00	
COMP	147	UG	Computing Theory	A-	4.000	14.80	
ECON	055	UG	Intro.Macroecon.:Theory,Policy	B	4.000	12.00	

Term Totals (Undergraduate)

	Attempt Hours	Passed Hours	Earned Hours	GPA Hours	Quality Points	GPA
Current Term:	16.000	16.000	16.000	16.000	52.00	3.25
Cumulative:	60.000	60.000	60.000	60.000	221.50	3.69

Unofficial Transcript

Term: Fall 2024

College: Sch of Engineering & Comp Sci
Major: Computer Science
Academic Standing: Good Standing
Additional Standing: Dean's Honor Roll

Subject	Course	Level	Title	Grade	Credit Hours	Quality Points	R
COMP	157	UG	Design/Analysis of Algorithms	A-	4.000	14.80	
ECPE	071	UG	Digital Design	A	3.000	12.00	
ECPE	170	UG	Computer Systems & Networks	B+	4.000	13.20	
ENGR	030	UG	Engr/Comput Ethics Society	A	3.000	12.00	
MHIS	005	UG	Music Appreciation	B	3.000	9.00	

Term Totals (Undergraduate)

	Attempt Hours	Passed Hours	Earned Hours	GPA Hours	Quality Points	GPA
Current Term:	17.000	17.000	17.000	17.000	61.00	3.58
Cumulative:	77.000	77.000	77.000	77.000	282.50	3.66

Unofficial Transcript

TRANSCRIPT TOTALS (UNDERGRADUATE) **-Top-**

	Attempt Hours	Passed Hours	Earned Hours	GPA Hours	Quality Points	GPA
Total Institution:	77.000	77.000	77.000	77.000	282.50	3.66
Total Transfer:	4.000	4.000	4.000	0.000	0.00	0.00
Overall:	81.000	81.000	81.000	77.000	282.50	3.66

Unofficial Transcript

COURSES IN PROGRESS **-Top-**

Term: Spring 2025

College: Sch of Engineering & Comp Sci
Major: Computer Science

Subject	Course	Level	Title	Credit Hours
COMP	135	UG	Human-Computer InterfaceDesign	3.000
COMP	173	UG	Operating Systems	4.000

COMP	175	UG	System Admin and Security	3.000
COMP	178	UG	Computer Network Security	3.000
ENGR	025	UG	Professional Practice Seminar	1.000

Unofficial Transcript

RELEASE: 8.7.1

© 2025 Ellucian Company L.P. and its affiliates.